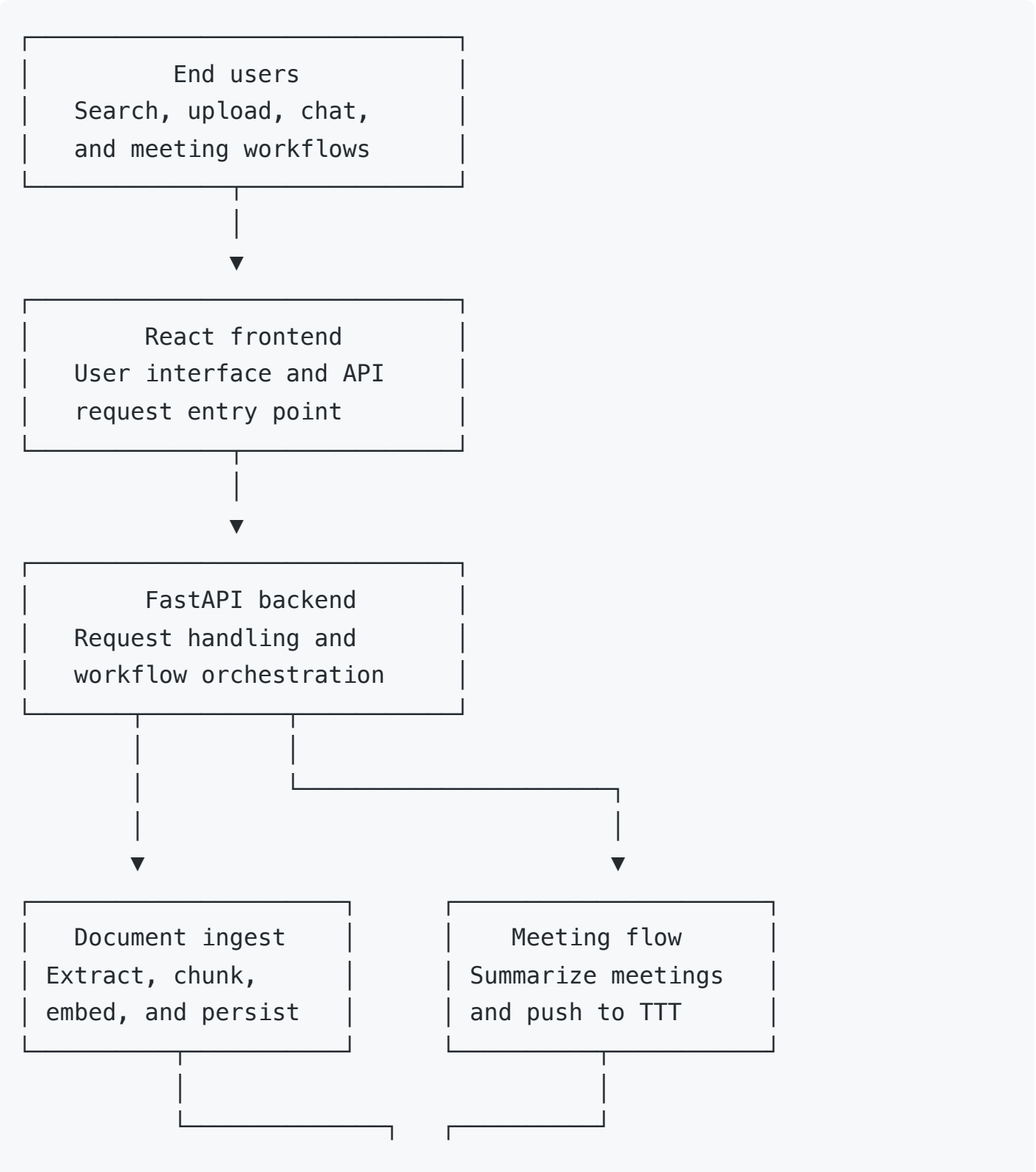
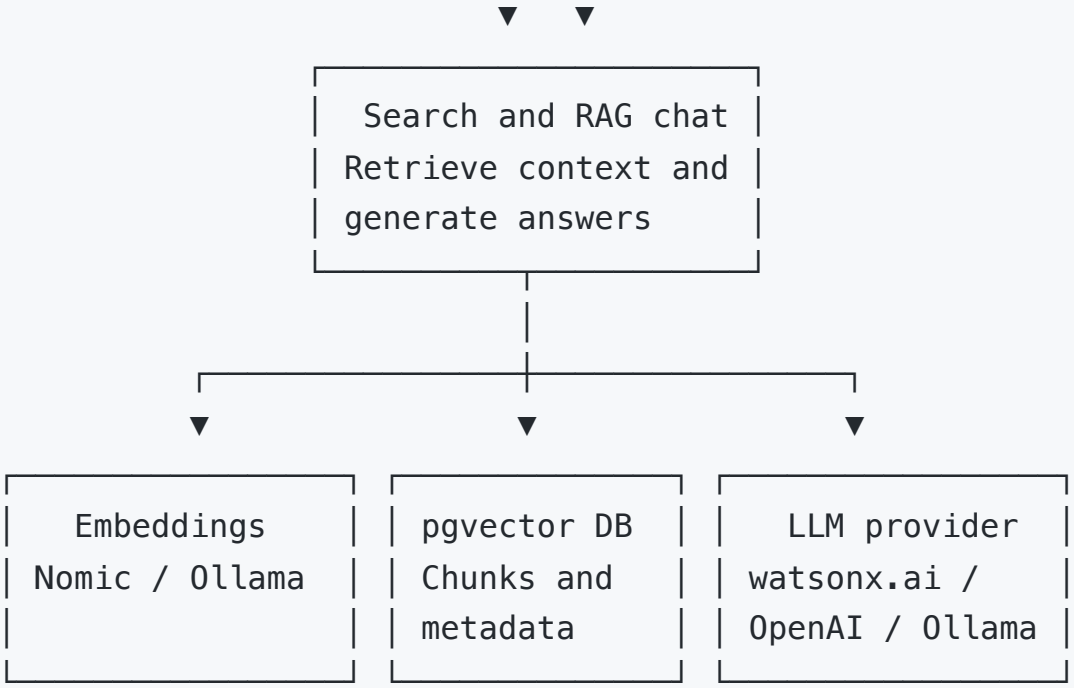
KnowledgeBase Overview

KnowledgeBase is a document and meeting intelligence platform that ingests source content, indexes it for semantic retrieval, supports grounded chat over enterprise knowledge, and converts meeting transcripts into structured summaries.

Platform Workflow





Workflow Summary

- **Frontend layer:** Collects user actions for upload, search, chat, and meeting workflows.
- **Backend layer:** Coordinates ingestion, retrieval, summarization, and downstream integrations.
- **Knowledge layer:** Uses embeddings, pgvector, and the active LLM provider to support retrieval and answer generation.

Key processes

1. Document Ingestion

Purpose

Prepare uploaded content for retrieval and downstream AI workflows.

How it works

1. Files are submitted through the UI or API.
2. `backend/kb/extractors.py` extracts text and splits content into chunks.
3. `backend/kb/embedder.py` generates embeddings for each chunk.
4. `backend/kb/ingest.py` stores chunks, vectors, and metadata in PostgreSQL with pgvector.

Result

The content becomes searchable and available as grounding context for chat.

2. Semantic Search

Purpose

Return relevant document content quickly without requiring full answer generation.

How it works

1. A user submits a search query.
2. The query is embedded with the configured embedding provider.
3. `backend/kb/search.py` runs vector similarity search against stored chunks.
4. The API returns the most relevant results with source context.

Result

Users can inspect relevant source material directly and move quickly to follow-up questions.

3. Retrieval-Augmented Chat

Purpose

Generate grounded answers based on indexed enterprise content.

How it works

1. A user asks a question in chat.
2. `backend/kb/chat.py` retrieves relevant chunks from the vector store.
3. The system builds a prompt with retrieved context and conversation history.
4. `backend/kb/llm.py` sends the request to the configured LLM provider.
5. The backend returns an answer with cited source references.

Result

Responses are grounded in indexed content rather than relying only on base model knowledge.

4. Meeting Summarization and Operationalization

Purpose

Turn meeting transcripts into searchable summaries and actionable records.

How it works

1. A transcript is uploaded through the meeting ingestion flow.
2. The content is processed through the same ingestion pipeline as other documents.
3. The backend generates a structured summary using the RAG workflow.
4. `backend/kb/pusher.py` sends the summary to Time Task Tracker when configured.

Result

Meeting information becomes both searchable and operationally useful.

Core System responsibilities

- **Frontend:** User interaction and workflow entry points via `frontend/src/App.jsx`
- **Backend API:** Request handling and orchestration via `backend/api.py`
- **Knowledge engine:** Extraction, embedding, retrieval, and chat orchestration via `backend/kb`
- **Vector store:** Persistent chunks, metadata, and embeddings in PostgreSQL with pgvector
- **LLM layer:** Configurable answer generation across local and cloud providers

Deployment Summary

KnowledgeBase supports a cloud-oriented deployment model with:

- a hosted React frontend
- a hosted FastAPI backend
- managed PostgreSQL with pgvector
- cloud embedding and LLM providers such as Nomic and `watsonx.ai`

For deeper implementation detail, see `ARCHITECTURE.md`.